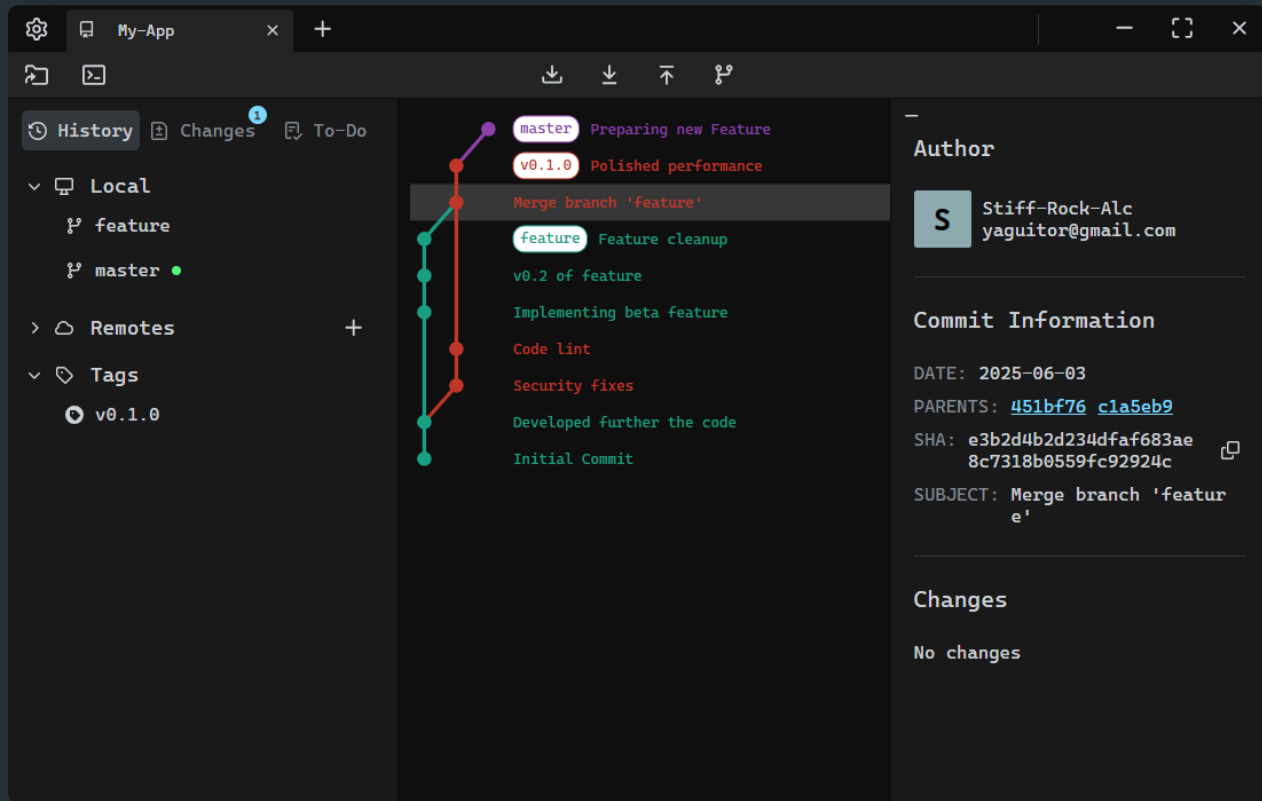


GitTaur - Una cliente de Git veloz y sencillo

GitTaur es una GUI para Git diseñada para aprovechar la velocidad y eficiencia de Rust y la experiencia moderna de usuario de React.



Créditos

Este proyecto no habría sido posible sin el gran trabajo de la comunidad de código abierto. Especial agradecimiento a los siguientes proyectos:

- [git2json por fabien0102](#) - Sin Licencia
Inspiró la lógica de conversión de los registros de Git al formato JSON. Esta funcionalidad ha sido completamente reimplementada en Rust para el backend de GitTaur.
- [gitgraph.js por nicoespeon](#) - Licencia MIT
Utilizado como base para el renderizado de gráficos en GitTaur. Se adaptó un fork personalizado para satisfacer las necesidades específicas de la aplicación.

- [auth-git2-rs por de-vri-es](#) - Licencia BSD-2-Clause

Proporcionó utilidades cruciales para manejar la autenticación en las operaciones del backend de `git2-rs`.

GitTaur utiliza muchos otros paquetes y crates de código abierto del ecosistema de Rust y JavaScript/React. Para una lista completa de todas las bibliotecas y licencias, puedes consultar [CREDITS.md](#).



Instalación de GitTaur

Para usuarios

GitTaur esta disponible para Windows 10 en adelante, para su correcto funcionamiento será necesario disponer de [Git](#) instalado en tu sistema, puesto que la aplicación interactúa con el para cualquier operación relacionada con los repositorios.

Existen 3 formas de obtener GitTaur:

- Instalador .msi: Puede requerir permisos de administrador.
- Instalador .exe: No requiere permisos de administrador, instala únicamente al usuario si se desea.
- Aplicación portable: No requiere instalación ninguna, se puede ejecutar directamente.

Elige la que más se adecue a tus necesidades.

Para desarrolladores

Si deseas ejecutar este proyecto en tu IDE, estos son los requisitos:

- Los archivos del proyecto, disponibles en el [repositorio oficial de GitTaur](#).
- [Microsoft C++ Build Tools](#) y [WebView2](#) (normalmente incluido en Windows) para el funcionamiento de Tauri.
- Compilador del lenguaje Rust, disponible en [rust-lang.org](#).
- Node.js para la ejecución de JavaScript/TypeScript, disponible en [nodejs.org](#).

Una vez cumplidos los requisitos, estos son los pasos a seguir para poder poner en funcionamiento el proyecto:

.git	03/06/2025 13:55	Carpeta de archivos	
.vscode	03/06/2025 13:55	Carpeta de archivos	
src	03/06/2025 13:55	Carpeta de archivos	
src-tauri	03/06/2025 13:55	Carpeta de archivos	
.gitignore	03/06/2025 13:55	Archivo de origen ...	1 KB
createCredits.js	03/06/2025 13:55	Archivo de origen ...	6 KB
CREDITS.md	03/06/2025 13:55	Archivo de origen ...	66 KB
deno.lock	03/06/2025 13:55	Archivo LOCK	36 KB
index.html	03/06/2025 13:55	Chrome HTML Do...	1 KB
LICENSE	03/06/2025 13:55	Archivo	2 KB
package.json	03/06/2025 13:55	Archivo de origen ...	2 KB
package-lock.json	03/06/2025 13:55	Archivo de origen ...	159 KB
README.md	03/06/2025 13:55	Archivo de origen ...	2 KB
tsconfig.json	03/06/2025 13:55	Archivo de origen ...	1 KB
tsconfig.node.json	03/06/2025 13:55	Archivo de origen ...	1 KB
vite.config.ts	03/06/2025 13:55	Archivo de origen ...	1 KB

Dirígete a la carpeta donde hayas descargado el proyecto y abre una terminal en la raíz del proyecto (deberías ver lo mismo que en la foto).

```
C:\Windows\system32\cmd.e: X + v
Microsoft Windows [Versión 10.0.22621.4317]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\yago.pernas\Desktop\GitTaur>npm install

added 298 packages, and audited 299 packages in 31s

133 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\yago.pernas\Desktop\GitTaur>
```

Ahora, desde la línea de comandos, ejecuta `npm install`, esto instalará las dependencias del frontend. Una vez finalice (como en la imagen) podrás ejecutar el programa desde la terminal con `npm run tauri dev`.

Ten en cuenta que la primera vez que ejecutes la aplicación puede llevar un rato, ya que automáticamente se descargarán todas las dependencias del backend.

Ahora ya puedes empezar a modificar el código como quieras. Los cambios del frontend (los archivos ubicados en `src`) deberían actualizarse automáticamente mientras tengas la aplicación ejecutando, en cualquier caso puedes refrescar la interfaz con `F5` o `Ctrl + Shift + R`. Para los cambios del backend (los archivos ubicados en `src-tauri`) tendrás que reiniciar la aplicación, por lo que deberás interrumpir la

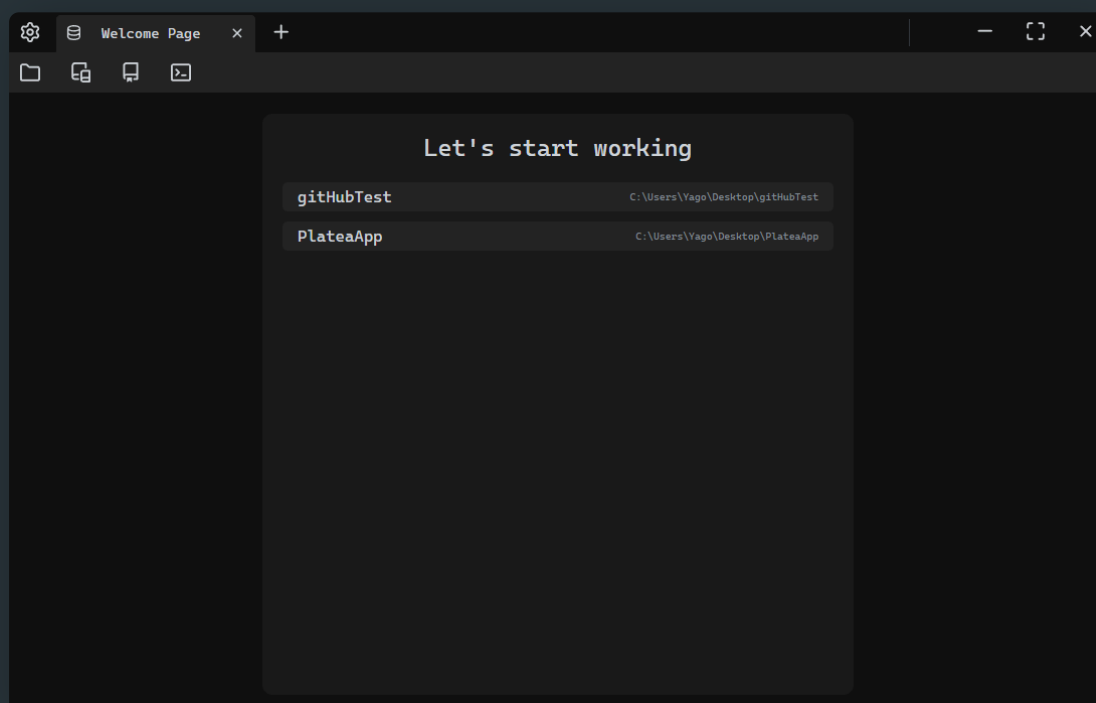
ejecución de la terminal pulsando `Ctrl + C` o cerrándola y abriendo otra y volver a ejecutar 'npm run tauri dev' desde la raíz del proyecto.



Características y uso

Pantalla de bienvenida

Esta es la primera pantalla que aparecerá al iniciar la aplicación por primera vez. Desde esta vista disponemos de acciones en forma de botones en la parte superior. En orden: Abrir repositorio, clonar repositorio, crear repositorio y abrir terminal. En la parte central podremos ver los repositorios que se hayan abierto recientemente y haciendo click izquierdo podremos abrirlo o podremos borrarlo o realizar otra acción sobre el haciendo click derecho.

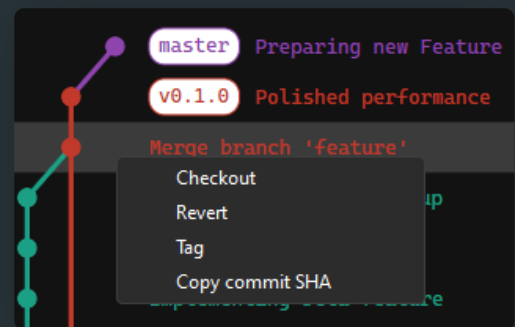


Visualización y gestión del historial del repositorio (Pestaña principal)

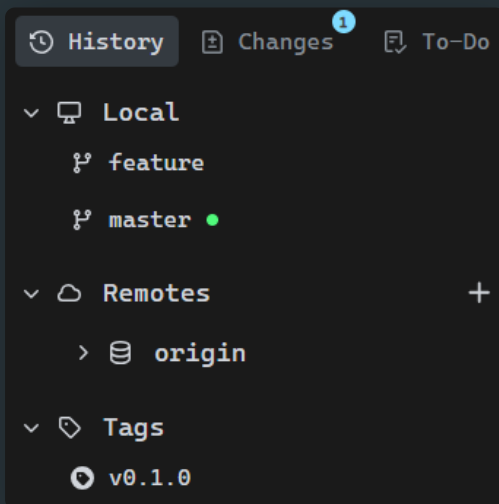


GitTaur provee un gráfico que representa el historial del repositorio. En el mismo podemos ver los commits, ramas y tags. Puedes hacer click derecho en cualquier commit para abrir el menú de acciones sobre ese commit.

Muchos de los elementos de la aplicación tienen acciones en el menú contextual. En el caso de un commit puedes encontrar las más fundamentales como checkout, revert, tag entre otras utilidades.



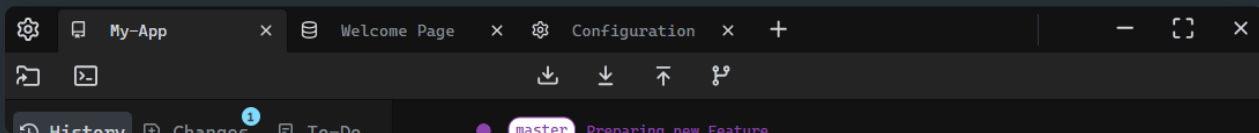
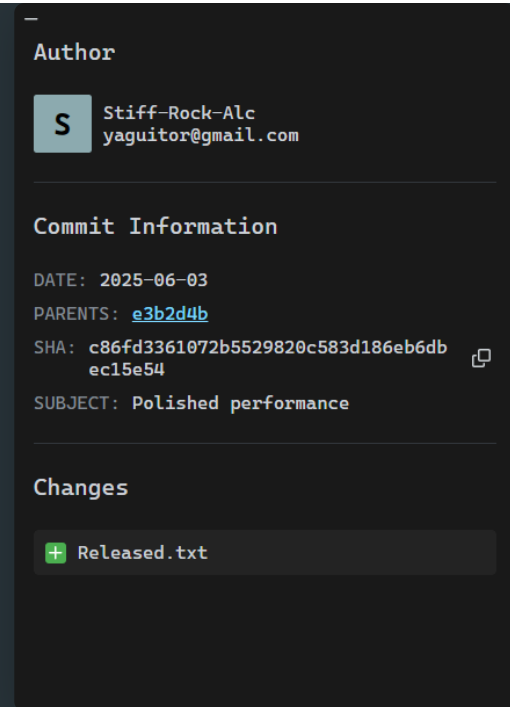
En el panel izquierdo hay dos secciones principales. En la parte superior tenemos tres botones para navegar entre las diferentes áreas de trabajo:



- **History:** Visualiza el historial del repositorio.
- **Changes:** Gestiona cambios en curso.
- **To-Do:** Abre un editor Markdown para listas de tareas.

Debajo hay tres menús desplegables: **Local:** Muestra ramas locales; **Remotes:** Remotos asociados con ramas desplegables; **Tags:** Etiquetas del repositorio. Cada elemento tiene un menú contextual para realizar operaciones. Este panel es fijo entre áreas de trabajo.

Cualquier commit que selecciones en el gráfico mostrará su información específica de manera estructurada y fácil de leer. Adicionalmente en la parte inferior se podrá ver que cambios fueron efectuados en ese commit.



En la parte superior encontramos los siguientes elementos empezando por la barra en la parte más superior de la aplicación:

- Botón de configuración: Abre una nueva pestaña para ajustar las preferencias relacionadas con la aplicación o con Git.
- Pestañas abiertas: Se muestran todas las pestañas abiertas ya sean repositorios, la pantalla de bienvenida o la configuración.
- Botones de control de la ventana: Controles básicos para gestionar la ventana (minimizar, maximizar, cerrar).

Debajo se sitúa la barra de acciones, la cual contiene diferentes botones que cambian en función de la pestaña en la que se encuentre el usuario.

Podemos encontrar acciones globales como:

-  Abre el repositorio en el explorador de archivos
-  Abre la terminal en el directorio del repositorio

O acciones de Git:

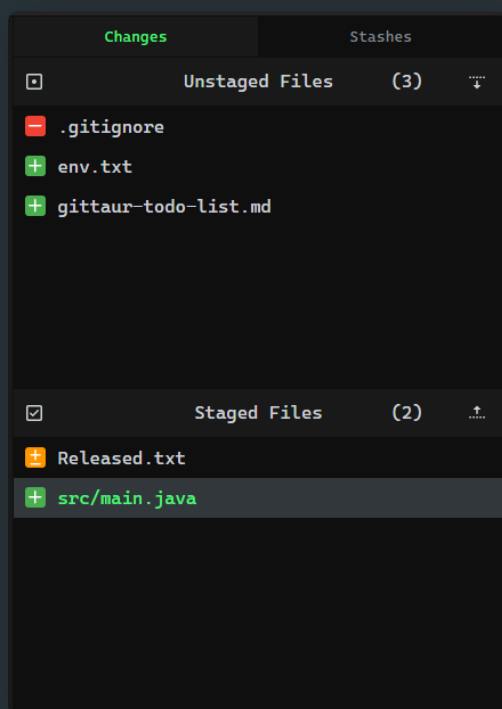
-  Efectúa un fetch al remoto que se indique en el pop-up que aparecerá

-  Aplica los nuevos cambios del remoto seleccionado al repositorio local si los hubiera
-  Sube los cambios del repositorio local al repositorio remoto seleccionado
-  Permite crear una rama a partir del commit en el que se encuentre el repositorio

Entre otros...

Administración de los cambios en curso

En el área de trabajo de "Changes" podremos ver, confirmar o descartar cambios que se hayan realizado en el proyecto desde el último commit o gestionar los stash creados.



En la parte central superior se ubican dos botones para cambiar el contenido entre los cambios locales o los stashes. Debajo tenemos dos contenedores. El superior muestra los cambios que no están preparados para incluir en un commit, mientras que el inferior muestra los que si están en el staging area.

En el panel derecho se muestra el contenido de los cambios del archivo seleccionado, indicando con un '+' las líneas añadidas y con un '-' las eliminadas y en la parte inferior encontramos dos

campos de texto y un botón para crear un nuevo commit con el resumen y el cuerpo escritos.

src/main.java

@@ -0,0 +1,3 @@

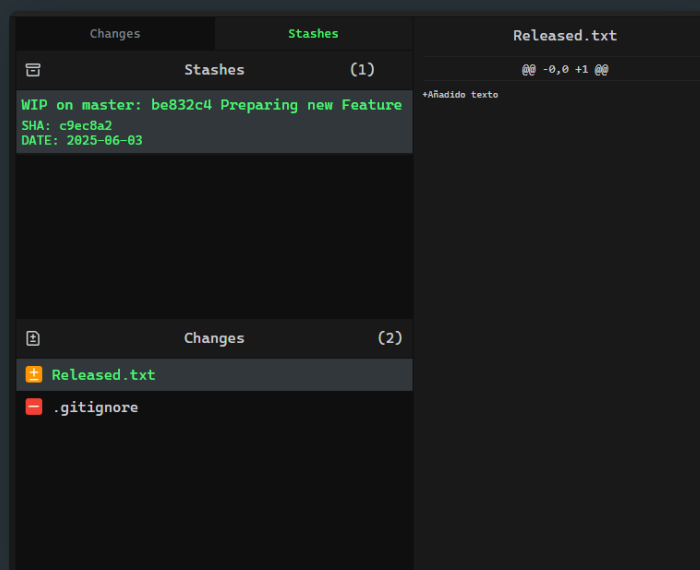
+public static void main(String args[]) {
+ System.out.println("Hello World!");
+}

Commit Options

Nuevos archivos15/50

Commit body (optional)

Commit changes



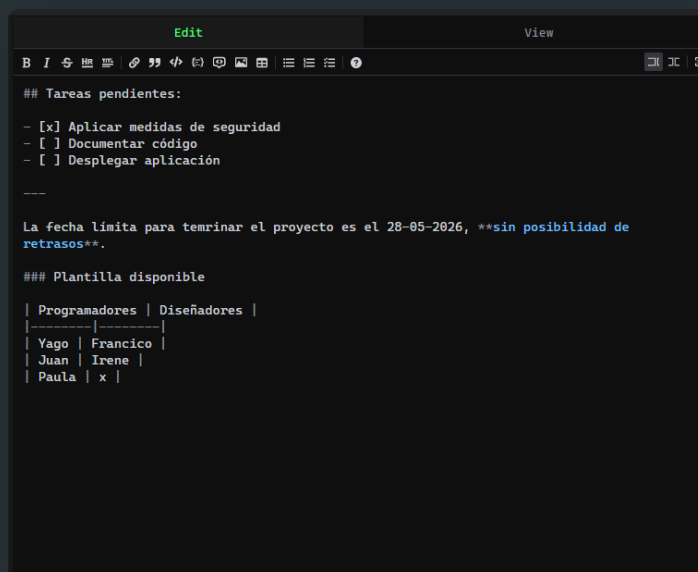
Si pulsamos el botón superior de 'Stashes' el contenido pasará a mostrar en la parte central los stashes creados, en la parte inferior los archivos que lo conforman y en el panel derecho muestra el contenido del archivo que se seleccione.

Creación listas de tareas del proyecto

La última área de trabajo permite al usuario crear un archivo de tipo `markdown` almacenado en la raíz de su proyecto. El contenido es totalmente libre, teniendo a disposición un editor y una vista previa para sacar partido de las capacidades de los archivos markdown, pudiendo hacer listas de tareas del proyecto, recordatorios, tablas,

esquemas o cualquier otro tipo de documento que pueda ayudar con proceso de desarrollo del proyecto.

En el editor se dispone de las mismas herramientas y capacidades que cualquier editor de texto pero con la sintaxis de markdown. También se puede mostrar la vista previa dividiendo el editor en dos o activar la pantalla completa usando los botones a la derecha del editor.



```
## Tareas pendientes:

- [x] Aplicar medidas de seguridad
- [ ] Documentar código
- [ ] Desplegar aplicación

---

La fecha límite para terminar el proyecto es el 28-05-2026, **sin posibilidad de retrasos**.

### Plantilla disponible

| Programadores | Diseñadores |
|-----|-----|
| Yago | Francico |
| Juan | Irene |
| Paula | x |
```



En la vista previa podemos ver el resultado del texto ya renderizado con la estructura que hayamos redactado en el editor.

Pantalla de configuración

Haciendo click en el icono del engranaje se abrirá la pestaña de configuración, donde podemos personalizar opciones relacionadas con características generales de la aplicación, su aspecto o configuraciones de Git.

General Configuration

Date format	YYYY-MM-DD
Max commits	1000
Create TodoList file	true ▼

Git Configuration

Username	Yago
Email	yaguitor@gmail.com
Clone directory	C:\Users\Yago\Documents 

Ui Configuration

Theme	System ▼
Accent Color	<input type="color" value="#00FF00"/>



Tecnologías usadas

GitTaur utiliza un stack tecnológico moderno que combina lenguajes y librerías ampliamente adoptados en la industria, como React y TypeScript en el frontend, junto con Tauri y Rust para el backend. Esta integración permite desarrollar una aplicación de escritorio eficiente, segura y fácil de mantener.

Tauri 2.0

Tauri es el componente principal de GitTaur. Es un framework que permite la creación de binarios y ejecutables para cualquier sistema operativo, tanto de escritorio como de dispositivos móviles. Fue desarrollado con Rust con el objetivo de crear ejecutables que pesen poco y que sus aplicaciones tengan un consumo eficiente de la memoria. Similar a otros frameworks como Electron, la interfaz gráfica de la aplicación se puede desarrollar usando cualquier tecnología web que compile a HTML, CSS y JavaScript para las aplicaciones de escritorio, y Swift o Kotlin para el desarrollo móvil, aprovechando los maduros ecosistemas alrededor de estas tecnologías.

Tauri provee canales de comunicación eficaces y fáciles de mantener entre el front y el back, diversidad de módulos llamados 'plugins' que complementan las funcionalidades de la aplicación (como cuadros de diálogo, lectura de archivos, sistemas de notificaciones, SQL, NFC, CLI, biometría y muchos más), y opciones de configuración de permisos y ámbito muy extensas para mejorar la seguridad de tu aplicación ajustando finamente sus capacidades.

React 18 + TypeScript - (Frontend)

React es una librería destinada a la creación de interfaces de usuario mediante JavaScript, dividiendo el documento HTML en subcomponentes de manera que permite recargar partes del documento sin tener que refrescar la vista entera, mejorando la experiencia de usuario y los tiempos de carga. Destaca por su enfoque declarativo y la facilidad de reutilizar componentes, lo cual mejora la mantenibilidad del código. TypeScript

complementa la funcionalidad de React frente al tradicional JavaScript gracias a su tipado estático, mejorando la seguridad del desarrollo. Ambos poseen ecosistemas maduros y amplias comunidades, lo cual facilita mucho el proceso de desarrollo.



Rust - (Backend)

Rust es un lenguaje de programación multiparadigma bastante reciente (fue anunciado en 2010). Se enfoca en lograr el rendimiento de un lenguaje como C o C++, pero garantizando la seguridad de memoria sin necesidad de un garbage collector, gracias a su sistema de propiedad. Se considera un lenguaje de bajo nivel, aunque no tan bajo como C (lenguaje con el que se integra fácilmente), puesto que no permite la gestión manual de la memoria. Por otro lado, también permite la escritura de código concurrente y multihilo de manera muy fiable y predecible. Es por esto que Rust es el principal lenguaje de backend que las aplicaciones de Tauri deberían usar.

Otras tecnologías usadas

- CSS: Estilos para la aplicación
- JSON: Archivos de guardado y configuración
- TOML: Configuración del proyecto

Código implementado

En este apartado vamos a abordar los aspectos más importantes del proyecto de manera general, explicando la estructura que compone a la aplicación y algunos ejemplos de código.

```
> .git
> .vscode
> dist
> docs
> node_modules
> src
> src-tauri
  .gitignore
  createCredits.js
  CREDITS.md
  deno.lock
  index.html
  LICENSE
  package-lock.json
  package.json
  README.md
  tsconfig.json
  tsconfig.node.json
  vite.config.ts
```

En la raíz del proyecto se encuentran diversos archivos y carpetas, la mayoría destinados a dependencias, configuraciones o documentación. Las dos carpetas más importantes son: `src`, que contiene el código del frontend desarrollado en React, y `src-tauri`, que alberga el backend implementado en Rust.

src/

En el apartado del frontend, la aplicación se organiza en varias carpetas que estructuran el código. Las principales son:

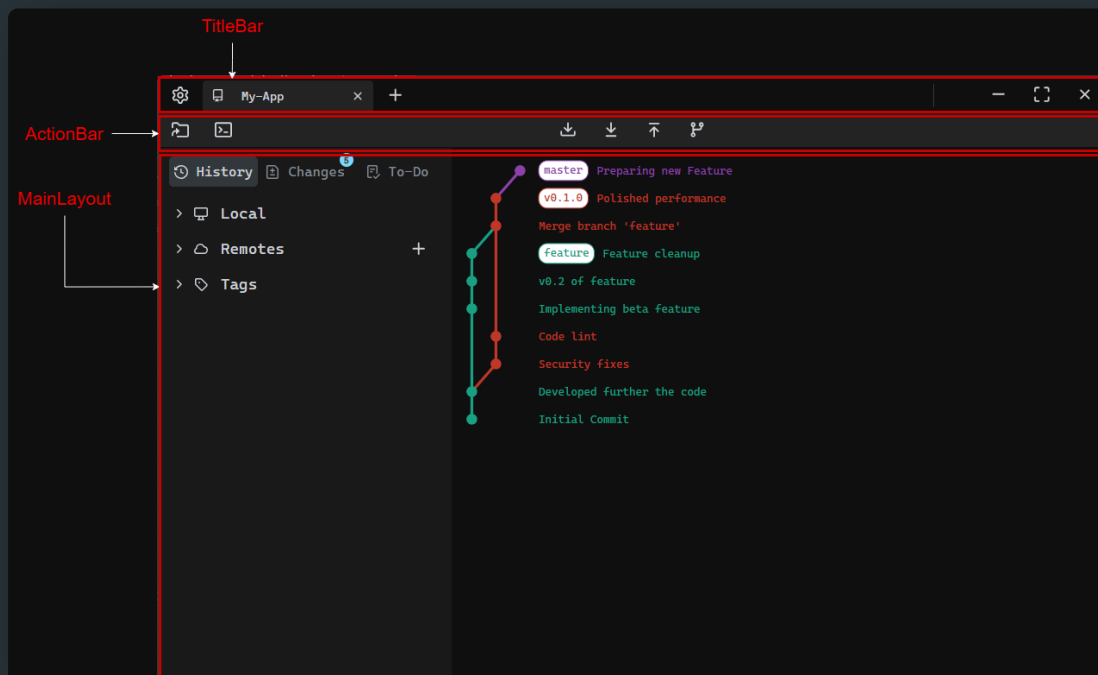
- **components:** Contiene los componentes principales de la aplicación, que a su vez se dividen en componentes en subcarpetas, encapsulando el comportamiento de la UI.
- **context:** Define los contextos de React para manejar estados globales.
- **types:** Incluye definiciones de tipos utilizados para la serialización y deserialización de datos entre el backend y el frontend, garantizando coherencia en la comunicación. Además,

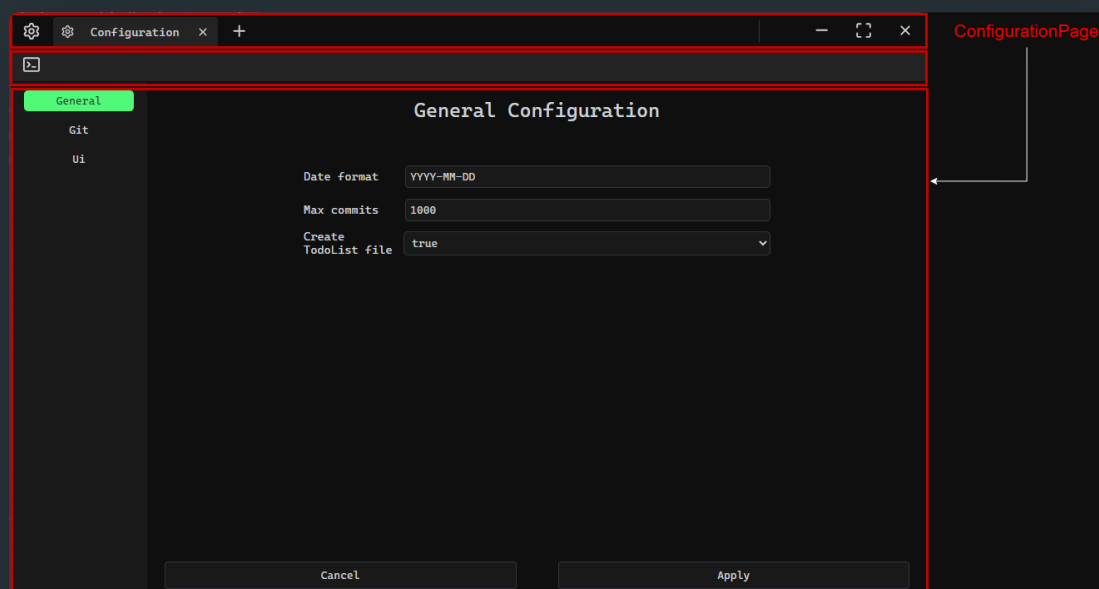
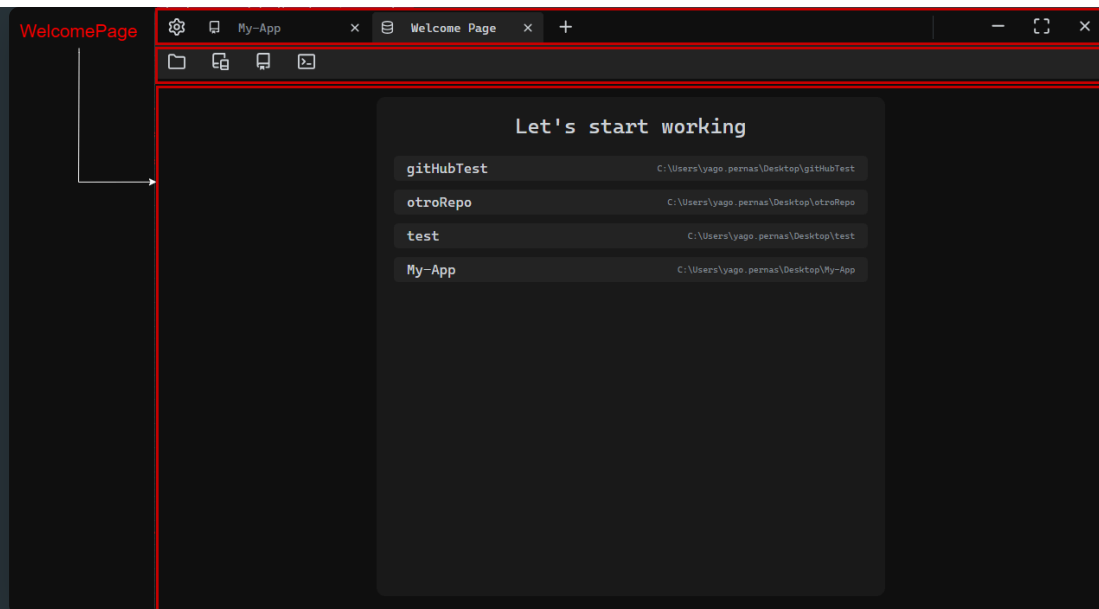
contiene algunos tipos exclusivos del frontend que sirven para estructurar y manejar datos específicos de la interfaz.

- **utils:** Funciones auxiliares que son utilizadas en diferentes partes de la aplicación.

```
src
├── assets
├── components
│   ├── ActionBar
│   ├── Common
│   ├── ConfigurationPage
│   ├── MainLayout
│   ├── TitleBar
│   └── WelcomePage
├── context
│   ├── AppContext.tsx
│   ├── GraphContext.tsx
│   ├── MainContext.tsx
│   └── PanelSyncContext.tsx
├── hooks
├── types
│   ├── appTabs.d.ts
│   ├── config.d.ts
│   ├── modals.d.ts
│   ├── repoInfo.d.ts
│   └── workspace.d.ts
├── utils
│   ├── App.tsx
│   ├── global.d.ts
│   ├── main.tsx
│   └── vite-env.d.ts
```

Podemos ver los principales componentes de la UI en estos esquemas:





El componente principal que une todos los elementos del frontend es `App.tsx`. Este componente actúa como el punto de entrada para la interfaz de usuario, organizando y gestionando los distintos elementos que conforman la aplicación. A continuación se muestra el código de `App.tsx`:

```
/* Imports */
```

```
function App() {
```

```
  /* Estados y efectos... */
```

```
  return (
```

tsx

```
<main className="container">
  <TitleBar />
  <ActionBar />
  <ToastContainer
    className="NotificationToast"
    position="top-right"
    autoClose={3000}
    newestOnTop
    hideProgressBar={true}
    pauseOnHover
    pauseOnFocusLoss
    transition={Zoom}
  />

  <WelcomePage />
  <ConfigPage />

  {workspace &&
    <PanelSyncProvider>
      {[...workspace.tabs].map(([key, _]) => (
        isType("Repo", key) && (
          <MainProvider key={key} repoPath={key}>
            <MainLayout
              key={key}
              isActive={workspace.activeTab === key}
            />
          </MainProvider>
        ))
      )}
    </PanelSyncProvider>
  }

  {activeModal === "confirmation" && <ConfirmationModal />}
</main>
);
}

export default App;
```

Desglose del componente `App.tsx`

1. Elementos persistentes:

- `<TitleBar />` y `<ActionBar />` son componentes que forman parte de la interfaz principal y permanecen visibles en todo momento.

2. Notificaciones:

- `<ToastContainer />` gestiona las notificaciones emergentes en la aplicación. Estas son temporales y desaparecen automáticamente después de un tiempo configurado (3000 ms en este caso).

3. Vista inicial y configuración:

- `<WelcomePage />` y `<ConfigPage />` son componentes que se muestran dependiendo del estado de la aplicación. Se autogestionan para ser visibles solo cuando es necesario.

4. Renderizado dinámico de repositorios:

- La última sección destaca por su gestión dinámica de vistas. Aquí se realiza un renderizado condicional basado en los repositorios abiertos por el usuario:
 - **Iteración de pestañas:** Se recorre cada repositorio abierto (`workspace.tabs`) y se genera un componente `<MainLayout>` para cada uno.
 - **Contextos adicionales:** Cada vista está envuelta en su propio contexto (`<MainProvider>`), lo que permite gestionar estados y lógica específicos para cada repositorio.
 - **Pestañas dinámicas:** Cada repositorio abierto se refleja en el `<TitleBar />` como una pestaña interactiva.

5. Modales:

- `<ConfirmationModal />` es un componente que se renderiza de manera condicional cuando el estado `activeModal` indica que debe mostrarse. Esto permite manejar interacciones específicas como confirmaciones de acciones.

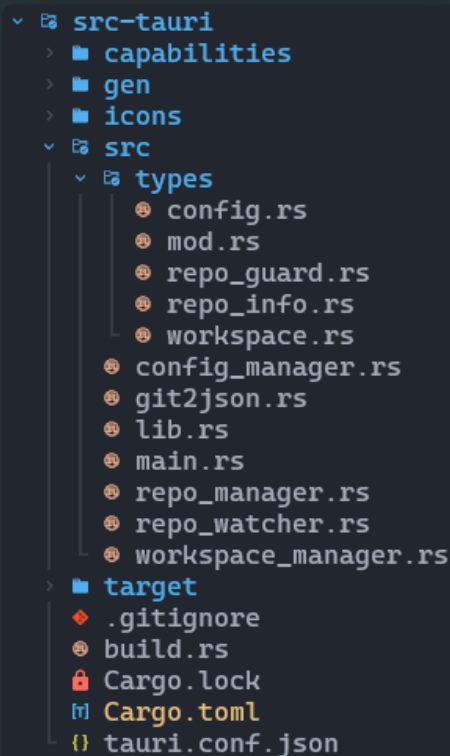
Resumen

El diseño de esta vista es solo un ejemplo de entre muchos otros componentes que utilizan mecanismos similares, entre otros:

- **Renderizado condicional:** Para mostrar u ocultar elementos en función del estado actual de la aplicación.
- **Contextos:** Proporcionando estados y lógica comunes entre componentes dentro del mismo.
- **Modularidad:** Dividiendo la interfaz en componentes reutilizables que gestionan su propia lógica y presentación.

Esta arquitectura permite flexibilidad, escalabilidad y dinamismo, mejorando la UX incluso en vistas complejas.

src-tauri/



En el backend, encontramos archivos de configuración junto con la carpeta `src`, que contiene el código principal del backend. Esta se divide en:

- **Raíz:** Archivos que definen la lógica principal del backend, como `main.rs` y otros módulos que implementan comportamientos clave.
- **types:** Carpeta que contiene definiciones de estructuras (`structs`), implementaciones (`impl`) y tipos utilizados para la comunicación entre el frontend y el backend.

El punto de entrada de la aplicación está en `main.rs`, posiblemente el archivo más breve de todos, ya que se compone sencillamente de una llamada a la función `run()` ubicada en `lib.rs`, donde realmente se encuentra la inicialización de la aplicación, el backend y los canales de enlace con el frontend.

```

main.rs

// Prevents additional console window on Windows in release, DO NOT REMOVE
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
  
```

```
gittaur_lib::run()
}
```

lib.rs

```
/* Imports y otras funciones de la app */

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    let run_result = tauri::Builder::default()
        /* Inicialización de plugins de Tauri */
        .plugin(tauri_plugin_opener::init())
        .plugin(tauri_plugin_shell::init())
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_os::init())
        /* Definición de las funciones que
        deben ser expuestas para poder ser
        llamadas desde el frontend */
        .invoke_handler(tauri::generate_handler![
            create_todo_file,
            save_todo_file,
            open_terminal,

            { /* ... */ }
        ])
        /* Acciones personalizadas a realizar
        durante la inicialización de la aplicación,
        manejando posibles errores que surjan de cada una
        (cargar configuraciones o pestañas previamente
        abiertas, iniciar el logger, etc.). */
        .setup(|app| {
            if let Err(e) = setup_logging(app) {
                handle_setup_error(e)?;
            }
            if let Err(e) = init_app_paths(app) {
                handle_setup_error(e)?;
            }
            if let Err(e) = set_app_globals(app) {
                handle_setup_error(e)?;
            }
            if let Err(e) = setup_app_theme(app) {
```

```

        handle_setup_error(e)?;
    }

    trace!("Setup finished!");

    Ok(())
})
.run(tauri::generate_context!());

/* Manejo de errores que puedan surgir durante la inicialización de la aplicación */
if let Err(e) = run_result {
    let msg = format!("Error while running GitTaur: {e}");
    error!("{msg}");
    tinyfiledialogs::message_box_ok(
        "GitTaur Error",
        &msg,
        tinyfiledialogs::MessageBoxIcon::Error,
    );
    std::process::exit(1);
}
}

```

Los archivos con el sufijo `_manager` encapsulan la lógica necesaria para gestionar aspectos funcionales clave de la aplicación. Estos ayudan a modularizar la lógica del programa y facilitan la organización del código. Entre los principales archivos se encuentran:

- `config_manager` : Gestiona la configuración de la aplicación, incluyendo la carga, actualización y almacenamiento de parámetros.
- `workspace_manager` : Maneja las pestañas abiertas y el estado del espacio de trabajo del usuario.
- `repo_manager` : Encargado de la interacción con los repositorios. Es vital para el funcionamiento del programa ya que implementa todas las funcionalidades relacionadas con el control de los repositorios. Utiliza la librería `git2-rs` para proporcionar un control de bajo nivel sobre los repositorios, permitiendo extraer información, modificar estados y añadir datos según sea necesario.

Ejemplo de una función en `repo_manager`

A continuación, se muestra un ejemplo del tipo de funcionalidad implementada en el archivo `repo_manager`. En este caso, la función `get_repo_info()` se encarga de recolectar información sobre el repositorio que el usuario abre para posteriormente mostrarla en la interfaz de usuario.

```
rust name=repo_manager.rs

/* Atributo que indica que esta función es un comando
que puede (y debe) ser expuesto al frontend */
#[command]
pub async fn get_repo_info(repo_path: String) -> Result<RepoInfo, String> {
    info!("Getting info of repo {}", repo_path);

    // Adquirimos el lock del repositorio, de esta manera aseguramos la c
    let _repo_lock = RepoGuard::new(&repo_path, true)?;

    /* Comprobamos que el string dado por parámetro
    se corresponde con un directorio de un repositorio*/
    if !is_repo(&repo_path, true)? {
        return Err(format!("{}", repo_path));
    }

    // Abrimos el repositorio (usando la librería git2-rs)
    let repo = Repository::open(&repo_path).map_err(|e| e.to_string())?;
    let name = repo_path.to_string();

    // Comprobamos la cabecera del repositorio
    let main_branch: String;
    if repo.head_detached().map_err(|e| e.to_string())? {
        main_branch = "master".to_string();
    } else {
        let head = repo.head().map_err(|e| e.to_string())?;
        if let Some(branch_name) = head.shorthand() {
            main_branch = branch_name.to_string();
        } else {
            let msg: &str = "Could not determine the current branch";
            error!("{}", msg);
            return Err(format!("Error: {}", msg));
        }
    }
}
```

```
// Obtenemos la rama activa actual
let current_branch = get_current_branch(&repo);

// Obtenemos una lista de las ramas locales
let local_branches = repo
    .branches(Some(git2::BranchType::Local))
    .map_err(|e| e.to_string())?
    .filter_map(|b| b.ok())
    .filter_map(|(b, _)| b.name().ok().flatten().map(|s| s.to_owned()))
    .collect::<Vec<String>>();

// Obtenemos una lista de las etiquetas del repositorio
let tags = repo
    .tag_names(None)
    .map_err(|e| e.to_string())?
    .iter()
    .filter_map(|t| t.map(|s| s.to_string()))
    .collect::<Vec<String>>();

// Obtenemos un HashMap de los remotos y las ramas de cada uno
let remotes: HashMap<String, Remote> = get_remote_branches(&repo)?;

/* Envolvemos la información en el objeto `RepoInfo`
para su serialización y envío al frontend */
let repo = RepoInfo {
    name,
    main_branch,
    current_branch,
    local_branches,
    remotes,
    tags,
};

// Devolvemos el resultado exitoso
Ok(repo)
}
```

Desde el frontend, se llamaría a esta función de la siguiente manera:


```
import { invoke } from "@tauri-apps/api/core";

function obtenerInfoRepositorio() {
  const directorio = "C:\\Users\\yago.pernas\\Documents\\Proyecto";

  invoke<RepoInfo>("get_repo_info", {repoPath: directorio})
    .then((value) => console.log("Información obtenida:" , value))
    .catch((e) => console.error("Error obteniendo información del repo"))
    .finally(() => { /* Ejecutar limpieza necesaria en caso de error */ })
}
```

Estos son solo algunos ejemplos de la estructura de código que se pueden encontrar en **GitTaur**. Siguiendo los estándares establecidos por Tauri, se garantiza que el proyecto sea coherente, claro y funcional. Además, esta estructura modular y bien organizada permite que la aplicación escale tanto en dimensión como en complejidad sin comprometer la claridad ni la mantenibilidad del código.